

Experimental Setup

> Dataset Details

- Training Dataset ~25k Images
- Test Dataset - 287 Images

> Hyperparameters Used for all experiments

- BATCH_SIZE = 32
- LEARNING_RATE = 1e-4
- NUM_EPOCHS = 15
- DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
- LR_STEP_SIZE = 1
- LR_GAMMA = 0.9
- NUM_WORKERS = 12

> Experiments Conducted

- I am using **one GPU P100** to train all these models. Training these models for longer epochs will help but because of resource crunch I will train it for **15 epochs**
- Same set of data augmentations were applied, early stopping mechanism was activated and cross entropy loss will be used for all experiments
- Global dataset is divide into train and validation data in **80:20 ratio**. Rest of the pipeline code is kept same for all experiments

Exp No.	Architecture	Train Data Used	Training Duration	Accuracy (Train, Val, Test)	Incorrect Classification in Test	Comments For Further Improvements
1.	CNN-Deep	40%	< 30 mins	(75, 78, 58)	121 images	Add more layers
2.	CNN-Deep	100%	~ 1 hour	(82, 83, 67)	94 images	Increase epochs (>15)
3.	CNN-Deep-SE	100%	~ 1 hour	(82, 85, 67)	96 images	Train for longer (> 15 epochs, try focal loss)
4.	CNN-Deep-CSE (Best Model)	100%	Epochs = 20 ~2.2 hours	(90, 91, 72)	81 images	Train for longer (> 30 epochs)

Defining the model yourself. So not use any off-the-shelf models here

Level 1: Add more layers to reach at least 85% accuracy on the validation dataset

Level 2: Investigate batch normalization and dropout to improve the model's performance

Level 3: Implement attention mechanisms to improve the model's performance

Key Improvements:

1. **More Layers:** Increased convolutional layers to enhance feature extraction.
2. **Batch Normalization:** Added after each convolution to stabilize learning and improve generalization.
3. **Dropout:** Applied after the first fully connected layer to reduce overfitting.
4. **Increased Fully Connected Layer Depth:** Added an intermediate layer to learn complex features.
5. **Implemented the modified CNN model with 2 types of** extension as mentioned below:
 - self-attention or squeeze-and-excitation (SE) blocks for better spatial attention.
 - Replaced the SE blocks with Convolutional Block Attention Module (CBAM), which includes both channel and spatial attention mechanisms. This should enhance feature extraction and improve classification accuracy.

Level 1. Create and use smaller dataset for faster training and smaller validation set for quick testing

Level 2. Merge two spectator classes into one

Level 3. Apply different transforms (which make sense) to the training and validation dataset (data augmentation). Apply them wisely for under-represented classes

Level 1: Created a Smaller Dataset for Faster Training

- Reduced the dataset size for quicker experimentation:
 - Used **40%** of the original dataset for training.
 - Used **10%** of the original dataset for validation.
- This allows for faster iteration without overloading computational resources.
- Adjusted the train_size and val_size accordingly to maintain an **80-20 split** within the smaller dataset.

Level 2: Merged Two Spectator Classes into One

- The original dataset had two separate classes: "spectators_long" and "spectators_short".
- These two classes were **merged into a single class "spectators"**.
- Used a **dictionary mapping approach** to relabel dataset targets before splitting into training and validation sets.

Level 3: Applied Different Data Augmentation Strategies

- **Training Set Augmentations (More Variation)**

- **Random Horizontal Flip** → Helps the model generalize to different viewpoints.
- **Random Rotation (15 degrees)** → Makes the model robust to orientation variations.
- **Color Jitter (brightness, contrast, saturation, hue changes)** → Handles different lighting conditions.
- **Resize and Normalize** → Standard preprocessing to ensure consistency.
- **Test / Validation Set Augmentations (Minimal Transformations)**
 - Only **resizing and normalization** were applied to ensure the evaluation remains accurate.

Final Adjustments

- After splitting, I explicitly **assigned the respective transforms to training and validation datasets**.
- **Created DataLoaders** with:
 - **Shuffling enabled for training** (helps prevent overfitting).
 - **No shuffling for validation** (ensures a stable evaluation set).

These modifications improve **training speed**, **model generalization**, and **class balance handling** while keeping computational costs reasonable.

Level 1. Implement early stopping based on validation accuracy

Level 2. Use tensorboard for visualization (show also in video)

Level 3. Evaluate 3 different criterion functions

Explanation of Criterion Functions:

- 1. CrossEntropyLoss:**
 - Used for multi-class classification problems.
 - Applies `log_softmax` internally and computes the negative log-likelihood loss.
 - Suitable for categorical classification tasks like ours.
- 2. Negative Log-Likelihood Loss (NLLLoss):**
 - Used with `log_softmax` output.
 - Requires explicit log probabilities from the model.
 - Less commonly used than `CrossEntropyLoss` unless working with already-logarithmically-transformed outputs.
- 3. Mean Squared Error Loss (MSELoss):**
 - Typically used for regression.
 - Not ideal for classification tasks since it assumes continuous targets.
 - Included here for experimentation, but likely not a good fit.
- 4. Focal Loss:**
 - Handles class imbalance better than `CrossEntropyLoss`.
 - Assigns higher weight to difficult samples by focusing more on misclassified examples.
 - Useful when some classes appear significantly more than others.

Level 1. Calculate test accuracy, and classification report (use sklearn.metrics for this). Fill the results dictionary with these metrics.

Level 2a. Calculate confusion matrix (use sklearn.metrics for this). Fill the results dictionary with this matrix.

Level 2b. Up to this point you've probably noticed, that, some data images are incorrectly labeled. Propose a strategy to automatically find these images and return a list of them to the user.

Level 3. Display the classification report and confusion matrix in a visually appealing way (e.g., using seaborn or matplotlib for confusion matrix visualization).

Completed this as mentioned. Will use these to compare 2 models. Check the code and visualizations added in the repo to learn more.

Level 1. Create/train two models in training phase. Change this code to evaluate both of them and compare results.

Level 2. Find existing exemplary image model (the best in your opinion for this task) and compare it (like in point 1) with the best one you trained

Level 3. Visualize last layer embeddings for the best trained model (you may need to install additional dependencies here). Visualize them to the user and mark the points denoted in part 2b of the previous assignment.

Check above table to learn about the experiment name and its setup. Here I will compare 2 models namely – CNN-Deep trained on 40% train-dataset and CNN-Deep trained with 100% train-dataset i.e., experiment 1 & 2 from the table. I have made the checkpoint files, different name for different experiments (so that all exp. can be compared) and shared that in code so please refer that:

Once you have correct version of tensorboard installed use this to display board in browser:

```
tensorboard --logdir./frame_classification/runs/" --port=6006
```

FAQ 1. What is a Confusion Matrix?

A **Confusion Matrix** is a table used to evaluate the performance of a classification model. It shows the actual vs. predicted classifications and provides insights into where the model is making errors. For a **binary classification** problem, the confusion matrix looks like this:

	Predicted Positive	Predicted Negative
Actual Positive	TP (True Positive)	FN (False Negative)
Actual Negative	FP (False Positive)	TN (True Negative)

For a **multi-class classification**, it extends to an $N \times N$ matrix where each row represents the actual class and each column represents the predicted class.

👉 Key Metrics Derived from Confusion Matrix:

- **Accuracy** = $(TP + TN) / (\text{Total Samples})$
- **Precision** = $TP / (TP + FP)$
- **Recall (Sensitivity)** = $TP / (TP + FN)$
- **F1 Score** = $2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$

📌 **Use Case:** Helps understand **misclassifications** and **class imbalance issues** in model predictions.

FAQ 2. What is a t-SNE 2D Plot for Last Embedding Layer?

t-SNE (t-Distributed Stochastic Neighbor Embedding) is a dimensionality reduction technique used for **visualizing high-dimensional data** in **2D or 3D space**.

Why Use t-SNE on the Last Embedding Layer?

The last embedding layer (often in deep learning models) represents the learned **feature space** just before classification. Plotting it with **t-SNE in 2D** helps:

- **Understand feature separability** → Are different classes well-separated?
- **Detect clusters** → Do similar samples form distinct groups?
- **Identify overlapping classes** → If two classes overlap a lot, the model might struggle with classification.
- **Debug model performance** → Misclassified points often lie near decision boundaries.

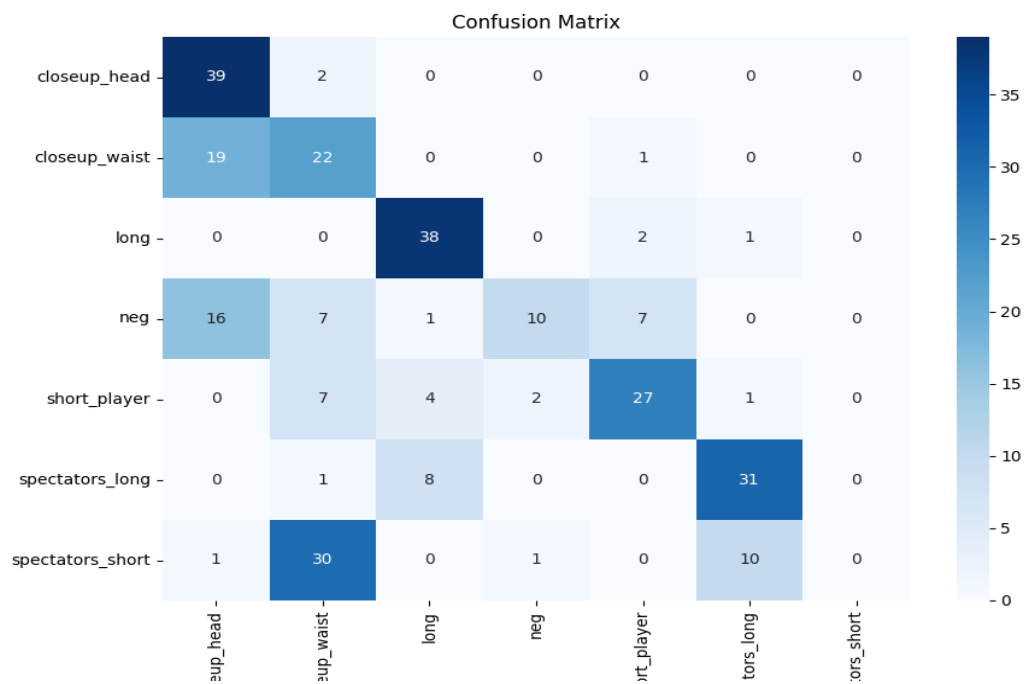
How to Generate a t-SNE Plot for Embeddings?

1. Extract the last **embedding layer's output** (before softmax).
2. Apply **t-SNE** to reduce dimensions to **2D or 3D**.
3. Plot with **matplotlib or seaborn**.

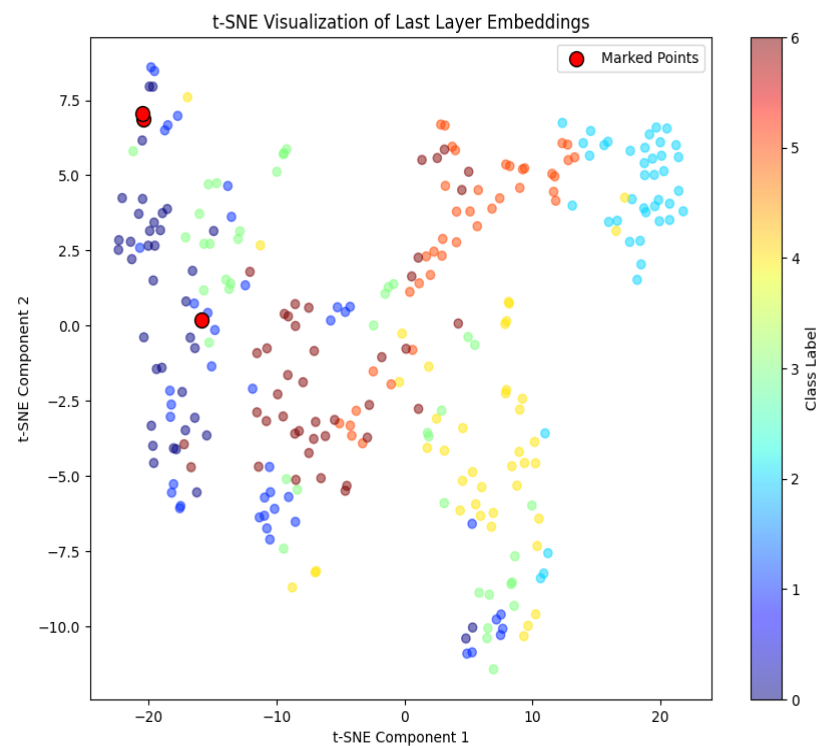
Interpretation:

- **Well-separated clusters** → Model has learned good feature representations.
- **Overlapping points** → Model struggles with class distinction.
- **Outliers** → Some samples might be misclassified or ambiguous.

Exp # 1. Confusion Metric



Exp # 1. TSNE Plot for last layer embedding visualization



Exp # 1. Training of the Model

```
Validation: 100%|██████████| 61/61 [00:17<00:00, 3.51it/s, val_loss=0.7641, val_acc=70.0]
Train Loss: 0.7634, Train Acc: 71.97%
Val Loss: 0.7641, Val Acc: 70.07%
```

```
Epoch 10/15
100%|██████████| 242/242 [01:17<00:00, 3.12it/s, loss=0.7451, acc=72.49%]
Validation: 100%|██████████| 61/61 [00:17<00:00, 3.48it/s, val_loss=0.6795, val_acc=74.5]
Train Loss: 0.7451, Train Acc: 72.49%
Val Loss: 0.6795, Val Acc: 74.52%
New best model saved! Val Acc: 74.52%
```

```
Epoch 11/15
100%|██████████| 242/242 [01:18<00:00, 3.10it/s, loss=0.7442, acc=72.62%]
Validation: 100%|██████████| 61/61 [00:17<00:00, 3.52it/s, val_loss=0.6794, val_acc=73.9]
Train Loss: 0.7442, Train Acc: 72.62%
Val Loss: 0.6794, Val Acc: 73.90%
```

```
Epoch 12/15
100%|██████████| 242/242 [01:17<00:00, 3.13it/s, loss=0.7067, acc=74.28%]
Validation: 100%|██████████| 61/61 [00:17<00:00, 3.53it/s, val_loss=0.6797, val_acc=76.5]
Train Loss: 0.7067, Train Acc: 74.28%
Val Loss: 0.6797, Val Acc: 76.54%
New best model saved! Val Acc: 76.54%
```

```
Epoch 13/15
100%|██████████| 242/242 [01:17<00:00, 3.12it/s, loss=0.7028, acc=74.30%]
Validation: 100%|██████████| 61/61 [00:17<00:00, 3.51it/s, val_loss=0.6481, val_acc=75.9]
Train Loss: 0.7028, Train Acc: 74.30%
Val Loss: 0.6481, Val Acc: 75.92%
```

```
Epoch 14/15
100%|██████████| 242/242 [01:17<00:00, 3.12it/s, loss=0.6911, acc=75.13%]
Validation: 100%|██████████| 61/61 [00:17<00:00, 3.50it/s, val_loss=0.6293, val_acc=77.9]
Train Loss: 0.6911, Train Acc: 75.13%
Val Loss: 0.6293, Val Acc: 77.94%
New best model saved! Val Acc: 77.94%
```

```
Epoch 15/15
100%|██████████| 242/242 [01:17<00:00, 3.12it/s, loss=0.6836, acc=74.95%]
Validation: 100%|██████████| 61/61 [00:17<00:00, 3.51it/s, val_loss=0.6378, val_acc=75.7]
Train Loss: 0.6836, Train Acc: 74.95%
Val Loss: 0.6378, Val Acc: 75.76%
```

Exp # 1. Testing of the Model

```
[29]: !python /kaggle/working/frame_classification/src/test.py --model_path "/kaggle/working/saved_models/best_model_cnndeep.pth"
```

```
/usr/local/lib/python3.10/dist-packages/torch/utils/data/dataloader.py:617: UserWarning: This DataLoader will create 12 worker processes in total. Our suggested max number of worker in current system is 4, which is smaller than what this DataLoader is going to create. Please be aware that excessive worker creation might get DataLoader running slow or even freeze, lower the worker number to avoid potential slowness/freeze if necessary.
```

```
warnings.warn(
```

```
Working with model cnndeep!
```

```
/kaggle/working/frame_classification/src/test.py:59: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In the future release, the default value for `weights_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
```

```
model.load_state_dict(torch.load(args.model_path, map_location=Config.DEVICE))
```

```
Starting evaluation ...
```

```
100%|██████████| 9/9 [00:15<00:00, 1.77s/it]
```

```
Completed evaluation! Reporting metrics ...
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/metrics/_classification.py:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
```

```
_warn_prf(average, modifier, msg_start, len(result))
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/metrics/_classification.py:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
```

```
_warn_prf(average, modifier, msg_start, len(result))
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/metrics/_classification.py:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
```

```
_warn_prf(average, modifier, msg_start, len(result))
```

```
Total incorrect images for cnndeep are: 121
```

```
Accuracy: 0.5798611111111112
```

```
Classification Report:
```

```
{'closeup_head': {'precision': 0.52, 'recall': 0.9512195121951219, 'f1-score': 0.6724137931034483, 'support': 41}, 'closeup_waist': {'precision': 0.3188405797101449, 'recall': 0.5238095238095238, 'f1-score': 0.39639639639639634, 'support': 42}, 'long': {'precision': 0.7450980392156863, 'recall': 0.926829268292683, 'f1-score': 0.8260869565217392, 'support': 41}, 'neg': {'precision': 0.7692307692307693, 'recall': 0.24390243902439024, 'f1-score': 0.37037037037037035, 'support': 41}, 'short_player': {'precision': 0.7297297297297297, 'recall': 0.6585365853658537, 'f1-score': 0.6923076923076923, 'support': 41}, 'spectators_long': {'precision': 0.7209302325581395, 'recall': 0.775, 'f1-score': 0.7469879518072289, 'support': 40}, 'spectators_short': {'precision': 0.0, 'recall': 0.0, 'f1-score': 0.0, 'support': 42}, 'accuracy': 0.5798611111111112, 'macro avg': {'precision': 0.5434041929206386, 'recall': 0.5827567612410818, 'f1-score': 0.5292233086438393, 'support': 288}, 'weighted avg': {'precision': 0.540121228178363, 'recall': 0.5798611111111112, 'f1-score': 0.5261683959214273, 'support': 288}}
```

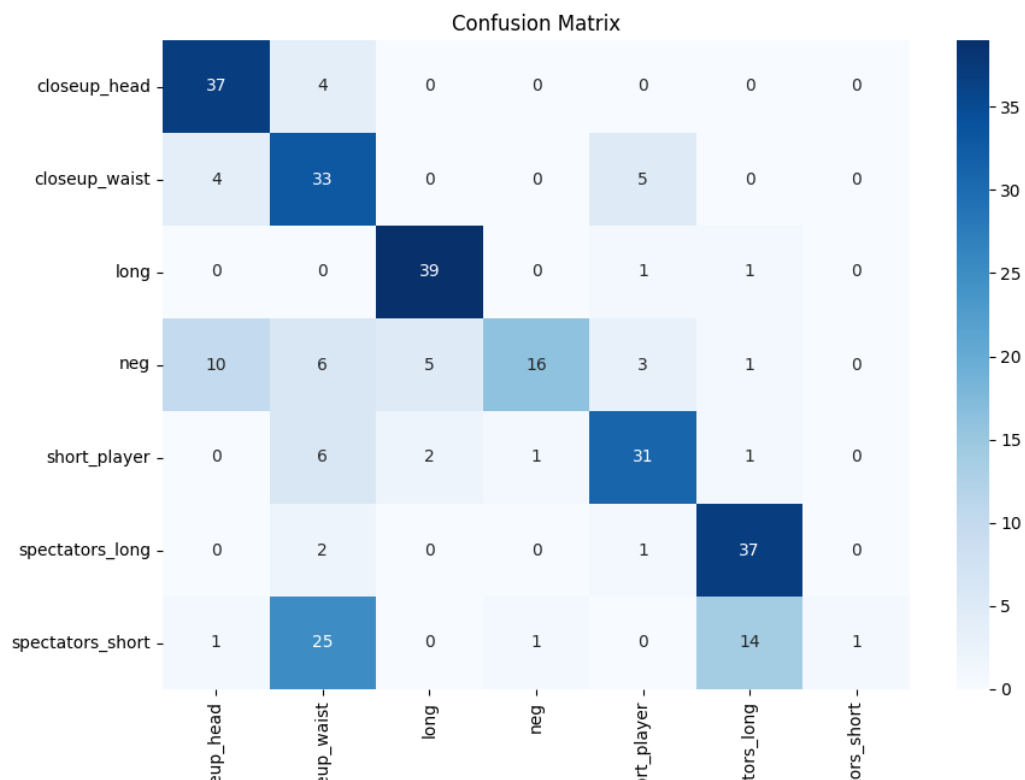
```
Figure(1000x700)
```

```
/usr/local/lib/python3.10/dist-packages/torch/utils/data/dataloader.py:617: UserWarning: This DataLoader will create 12 worker processes in total. Our suggested max number of worker in current system is 4, which is smaller than what this DataLoader is going to create. Please be aware that excessive worker creation might get DataLoader running slow or even freeze, lower the worker number to avoid potential slowness/freeze if necessary.
```

```
warnings.warn(
```

```
Figure(1000x800)
```


Exp # 1. Confusion Metric (better than experiment 1 when compared cell by cell)



Exp # 1. Training of the Model

```
Epoch 9/15
100%|██████████| 604/604 [03:16<00:00, 3.08it/s, loss=0.5945, acc=78.69%]
Validation: 100%|██████████| 151/151 [00:43<00:00, 3.46it/s, val_loss=0.5238, val_acc=81.27%]
Train Loss: 0.5945, Train Acc: 78.69%
Val Loss: 0.5238, Val Acc: 81.27%
New best model saved! Val Acc: 81.27%

Epoch 10/15
100%|██████████| 604/604 [03:14<00:00, 3.11it/s, loss=0.5754, acc=79.11%]
Validation: 100%|██████████| 151/151 [00:43<00:00, 3.46it/s, val_loss=0.4994, val_acc=81.85%]
Train Loss: 0.5754, Train Acc: 79.11%
Val Loss: 0.4994, Val Acc: 81.85%
New best model saved! Val Acc: 81.85%

Epoch 11/15
100%|██████████| 604/604 [03:15<00:00, 3.08it/s, loss=0.5536, acc=80.15%]
Validation: 100%|██████████| 151/151 [00:43<00:00, 3.48it/s, val_loss=0.4832, val_acc=82.70%]
Train Loss: 0.5536, Train Acc: 80.15%
Val Loss: 0.4832, Val Acc: 82.70%
New best model saved! Val Acc: 82.70%

Epoch 12/15
100%|██████████| 604/604 [03:13<00:00, 3.11it/s, loss=0.5446, acc=80.45%]
Validation: 100%|██████████| 151/151 [00:43<00:00, 3.49it/s, val_loss=0.4684, val_acc=82.70%]
Train Loss: 0.5446, Train Acc: 80.45%
Val Loss: 0.4684, Val Acc: 82.70%

Epoch 13/15
100%|██████████| 604/604 [03:14<00:00, 3.11it/s, loss=0.5352, acc=80.77%]
Validation: 100%|██████████| 151/151 [00:43<00:00, 3.48it/s, val_loss=0.4691, val_acc=83.26%]
Train Loss: 0.5352, Train Acc: 80.77%
Val Loss: 0.4691, Val Acc: 83.26%
New best model saved! Val Acc: 83.26%

Epoch 14/15
100%|██████████| 604/604 [03:14<00:00, 3.11it/s, loss=0.5231, acc=80.91%]
Validation: 100%|██████████| 151/151 [00:43<00:00, 3.45it/s, val_loss=0.4665, val_acc=83.12%]
Train Loss: 0.5231, Train Acc: 80.91%
Val Loss: 0.4665, Val Acc: 83.12%

Epoch 15/15
100%|██████████| 604/604 [03:15<00:00, 3.09it/s, loss=0.5197, acc=81.43%]
Validation: 100%|██████████| 151/151 [00:43<00:00, 3.47it/s, val_loss=0.4566, val_acc=83.03%]
Train Loss: 0.5197, Train Acc: 81.43%
Val Loss: 0.4566, Val Acc: 83.03%
```


Exp # 1. Testing of the Model

Level 1: Add code to analyze frame_numpy by the model, handle timestamps and predictions in the way it will allow to return JSON with scene segments (see the doc)

Level 2a: Implement error handling for cases when the model is not available or the input frame is not recognized.

Level 2b: Implement a stride in the frame analysis to improve performance. Use two frames per second.

Level 3: Implement a mechanism to detect scene transitions between segments. Use the OpenCV library for this.

ALL Levels: Implement at least 3 unit tests for this module. Note that you may choose the unit test framework you're most familiar with. Install it in docker

Implemented Features

Level 1: Frame Analysis & JSON Output (check dummy json output below)

- Loads and applies the model to each processed frame.
- Stores **timestamps, predictions, and class probabilities**.
- Returns a structured **JSON output** with **scene segments**.

Level 2a: Error Handling

- Catches errors when loading the model (`torch.load`).
- Handles video loading failures (`av.open`).
- Skips frames if the model is unavailable.

Level 2b: Stride for Performance Optimization

- Processes **only 2 frames per second** instead of all frames.
- Uses `frame_nb % stride == 0` to implement skipping.

Level 3: Scene Transition Detection

- Uses **OpenCV's frame difference method** to detect **scene changes**.
- Starts a **new segment** when a transition is detected.

```
{
  "segments": [
```

```

{
  "start_time": 0,
  "end_time": 5.32,
  "predictions": [
    {
      "time": 0.00,
      "class_id": 2,
      "probabilities": [[0.1, 0.3, 0.5, 0.1]]
    },
    {
      "time": 0.50,
      "class_id": 2,
      "probabilities": [[0.2, 0.4, 0.3, 0.1]]
    }
  ]
},
{
  "start_time": 5.32,
  "end_time": 10.12,
  "predictions": [
    {
      "time": 5.50,
      "class_id": 1,
      "probabilities": [[0.3, 0.2, 0.4, 0.1]]
    }
  ]
}
]
}

```

Summary

- **Installed pytest in Docker**
- **Implemented 3 unit tests:**
 1. **Model loading** (handles missing or incorrect models)
 2. **Frame analysis** (checks processing of images)
 3. **Scene transition detection** (compares frame differences)
- **Dockerized the test environment** for easy execution

Detecting scene transitions in a video is an important aspect of video analysis, particularly in applications such as video editing, surveillance, and content summarization. OpenCV provides useful functions to implement techniques like Structural Similarity Index (SSIM) and frame difference to identify these transitions.

Scene Transition Detection Techniques

1. Frame Differencing:

- This technique is based on comparing consecutive frames in a video. A significant change in pixel values between two frames may indicate a scene transition.
- You typically compute the absolute difference between the frames, convert it to grayscale, and then threshold it to determine whether the difference is significant.

2. SSIM (Structural Similarity Index):

- SSIM is a perceptual metric that quantifies the visual impact of differences between two images. Unlike frame differencing, which uses absolute differences, SSIM considers changes in structural information, luminance, and contrast.
- An SSIM value close to 1 indicates that the frames are similar, while values significantly less than 1 indicate a notable change.

The provided text explains how to use the PyAV library in Python to handle video files. It breaks down a code snippet into key components:

1. **Opening a Media File:** The code opens a video file from a specified URL, allowing for various configurations using options.
2. **Decoding the Video Stream:** A generator is created to yield frames from the first video stream, permitting frame-by-frame processing.
3. **Getting Frame Rate:** The average frames per second (FPS) of the video stream is retrieved, which is crucial for understanding playback speed or for applications like time-lapse analysis.
4. **Calculating the Stride:** The stride is defined as half the average frame rate, enabling processing of every second frame to reduce computational load while still capturing important content.

Video file is opened, frames are decoded, and every 15th frame (if the frame rate is 30 FPS) is processed. This skipping of frames helps in managing resources efficiently while analyzing larger videos.